# سمافور و مانیتور: مطالعه تطبیقی اثبات‌شده تمرین ۴ — دکر، پترسون،

انحصار متقابل با اثبات کامل روی همه درهم‌تنیدگی‌ها و اندازه‌گیری روی نخ‌های واقعی

**عنوان تمرین**دکر، پترسون، سمافور و مانیتور
**درس**سیستم‌عامل پیشرفته
**موضوع**انحصار متقابل و همگام‌سازی فرایندها
**روش اثبات**بررسی کامل فضای حالت روی همه درهم‌تنیدگی‌ها
**الگوریتم‌های بررسی‌شده**۸ مورد (۴ درست، ۳ عمداً معیوب، ۱ سخت‌افزاری)
**مکانیزم‌های اندازه‌گیری‌شده**دکر، پترسون، TAS، سمافور، مانیتور و ۳ گونه خواننده/نویسنده
**قالب تحویل**گزارش فارسی و انگلیسی (Word و PDF) + کد اجرایی + تست
**مخزن پروژه**github.com/kaspian021/Multilevel-Queue-Scheduling
**شاخه تمرین**Exercise-4-Dekker-Peterson-Semaphore-Monitor

# فهرست مطالب


| | |
|---|---|
| چکیده، کلیدواژه‌ها و نقشه گزارش | 3 |
| مقدمه، بیان مسئله و اهداف | 4 |
| سه ویژگی و تفاوت آن‌ها | 5 |
| چرا اجرای برنامه هیچ چیزی را اثبات نمی‌کند | 6 |
| بررسی‌گر مدل چگونه کار می‌کند | 7 |
| الگوریتم دکر | 8 |
| دکر: نتیجهٔ بررسی‌شده | 9 |
| الگوریتم پترسون | 10 |
| چرا فضاهای حالت با هم فرق دارند | 11 |
| گونهٔ معیوب ۱: متغیر قفل تکی | 12 |
| گونهٔ معیوب ۲: اعلام و سپس چرخیدن | 13 |
| اعلام-و-سپس-چرخیدن: چرا ترتیب دو گام مهم است | 14 |
| انحصار درست، انتظار نامحدود :Test-and-Set | 15 |
| سمافورها | 16 |
| سمافورها: نتیجهٔ بررسی‌شده | 17 |
| گونهٔ معیوب ۳: انتظار دوره‌ای | 18 |
| بن‌بست: چهار شرط و راه‌های شکستن آن‌ها | 19 |
| راهبردهای پیشگیری از بن‌بست | 20 |
| مانیتورها | 21 |
| اندازه‌گیری: مانیتور بافر محدود | 22 |
| بافر محدود: هزینهٔ بررسی دوباره | 23 |
| مسئلهٔ خوانندگان و نویسندگان | 24 |
| خوانندگان و نویسندگان: خواندن نتایج | 25 |
| درستی چه هزینه‌ای دارد | 26 |
| تفسیر اعداد توان | 27 |
| رفتار در شرایط رقابت | 28 |
| هزینهٔ پنهان چرخیدن | 29 |
| جدول مقایسه | 30 |
| جدول مقایسه به صورت ماتریس ویژگی‌ها | 31 |
| چه زمانی از چه چیزی استفاده کنیم | 32 |
| محدودیت‌ها و دامنهٔ صادقانه | 33 |
| نتیجه‌گیری و پاسخ‌ها | 34 |
| منابع و دستورهای بازتولید | 35 |
| فهرست فایل‌ها | 36 |
| فهرست شکل‌ها و بازتولید کامل | 37 |

# چکیده، کلیدواژه‌ها و نقشه گزارش

## چکیده

گام‌های اتمی است؛ و دقیقاً همین کار توسط بررسی‌گر مدلی انجام می‌شود که برای این تمرین ساخته شده است. و موفقیت‌آمیز بودن آن هیچ چیزی را اثبات نمی‌کند. آنچه اثبات می‌کند، گزاره‌ای دربارهٔ همهٔ درهم‌تنیدگی‌های ممکن الگوریتم دکر، الگوریتم پترسون، سمافور و مانیتور. نکته روش‌شناختی اصلی این است که اجرای یک برنامه هم‌روند در این تمرین چهار رویکرد به انحصار متقابل بررسی شده است:

کامل اند و نه نمونه‌برداری‌شده. در کنار اثبات‌ها، 5 مکانیزم روی نخ‌های واقعی اجرا و هزینهٔ درستی اندازه‌گیری شد. و سه گونه عمداً معیوب. بزرگ‌ترین فضای حالت تنها 324 حالت دارد، بنابراین همهٔ نتایج این گزارش شد: چهار مورد درست (دکر، پترسون، سمافور دودویی و مانیتور Mesa)، یک پریمتیو سخت‌افزاری (Test-and-Set) هشت الگوریتم بررسی

## کلیدواژه‌ها

متغیر شرطی؛ معنای Mesa؛ انتظار محدود؛ گرسنگی؛ بن‌بست؛ انتظار دوره‌ای؛ بررسی مدل؛ پیمایش فضای حالت. انحصار متقابل؛ بخش بحرانی؛ الگوریتم دکر؛ الگوریتم پترسون؛ Test-and-Set؛ سمافور؛ مانیتور؛

## نقشه گزارش

- صفحات ۳ تا ۶: مسئله، سه ویژگی و روش کار.
- صفحات ۷ و ۸: بررسی‌گر مدل و نحوهٔ خواندن یک شاهد.
- صفحات ۹ تا ۱۴: دکر، پترسون و دو پروتکل نرم‌افزاری معیوب.
- صفحات ۱۵ تا ۱۸: Test-and-Set، سمافور و انتظار دوره‌ای.
- صفحات ۱۹ تا ۲۲: مانیتور، بافر محدود و مسئلهٔ خواننده/نویسنده.
- صفحات ۲۳ تا ۲۶: هزینهٔ اندازه‌گیری‌شده، رقابت و جدول مقایسه.
- صفحات ۲۷ تا آخر: محدودیت‌ها، نتیجه‌گیری، منابع و فهرست فایل‌ها.

# مقدمه، بیان مسئله و اهداف

## بیان مسئله

مورد پروتکل نرم‌افزاری دست‌نویس برای دقیقاً دو شرکت‌کننده، یکی پریمتیو سیستم‌عامل و یکی انتزاع در سطح زبان. نباید بیش از یک نخ هم‌زمان آن را اجرا کند. چهار مکانیزم این تمرین این مسئله را در سطوح مختلف حل می‌کنند: دو نخ‌های هم‌روند داده‌های مشترک دارند. بخش بحرانی قطعه‌کدی است که به وضعیت مشترک دست می‌زند و

## اهداف

- بیان سه ویژگی‌ای که یک راه‌حل ممکن است داشته باشد یا نداشته باشد.
- توضیح اینکه چرا یک اجرای موفق تست هیچ چیزی را اثبات نمی‌کند.
- ساخت یک بررسی‌گر مدل کامل و استفاده از آن روی هشت الگوریتم.
- نمایش شکست یک پروتکل معیوب با یک ردیابی شاهد مشخص.
- پیاده‌سازی سمافور و مانیتور از پایه و اندازه‌گیری آن‌ها.
- بازتولید قطعی یک بن‌بست و نشان دادن راه پیشگیری از آن.
- اندازه‌گیری هزینهٔ هر مکانیزم در شرایط رقابت.

## قواعد مدل در سراسر گزارش

- دو (یا بیشتر) نخ، یک ناحیهٔ متغیر مشترک، بدون خرابی.
- خواندن و نوشتن یک کلمهٔ مشترک اتمی است.
- حافظه به‌صورت ترتیبی سازگار است: کامپایلر و پردازنده بازمرتب نمی‌کنند.
- نخی که در بخش غیربحرانی است سرانجام دوباره درخواست ورود می‌دهد.
- زمان‌بندی ضعیفاً منصفانه است: نخی که می‌تواند اجرا شود سرانجام اجرا می‌شود.

**چرا این مهم است:** به کار روند: روی پردازندهٔ امروزی، بدون موانع حافظهٔ صریح، درستی آن‌ها اصلاً تضمین نمی‌شود. فرض سازگاری ترتیبی است که باعث می‌شود دکر و پترسون بیشتر به عنوان نتایج تاریخی مطالعه شوند تا در عمل همین

# سه ویژگی و تفاوت آن‌ها

## انحصار متقابل

نباشند. این یک ویژگی ایمنی است: یک درهم‌تنیدگی بد آن را می‌شکند، و یک ردیابی شاهد برای رد آن کافی است. هیچ دو نخی هم‌زمان در بخش بحرانی

## پیشرفت (آزادی از قفل‌شدگی)

را برقرار کند و باز هم اینجا شکست بخورد: دو نخ می‌توانند با ادب متقابل یکدیگر را برای همیشه متوقف کنند. بحرانی نیست و کسی می‌خواهد وارد شود، انتخابِ ورود برای همیشه عقب نیفتد. یک راه‌حل می‌تواند انحصار متقابل اگر هیچ نخی در بخش

## انتظار محدود (آزادی از گرسنگی)

انصاف است و از پیشرفت قوی‌تر است. Test-and-Set دو ویژگی نخست را دارد و در این یکی شکست می‌خورد. بر تعداد دفعاتی که یک نخ متقاضی می‌تواند پشت سر گذاشته شود وجود داشته باشد. این ویژگی زنده‌مانی دربارهٔ کرانی

| برای رد آن چه لازم است | توسط چه چیزی نقض می‌شود | نوع | ویژگی |
|---|---|---|---|
| یک ردیابی شاهد | یک درهم‌تنیدگی بد | ایمنی | انحصار متقابل |
| یک حالت گیرکردهٔ دست‌یافتنی | ادب متقابل | زنده‌مانی | پیشرفت |
| یک دور دست‌یافتنی منصفانه | سبقت بی‌کران | زنده‌مانی | انتظار محدود |

جدول 1. سه ویژگی و آنچه برای رد هر یک لازم است

## یک پرسش چهارم و جدا: بن‌بست

به‌تنهایی کاملاً درست باشد و باز هم در اثر استفادهٔ نادرست بن‌بست بسازد. صفحهٔ ۱۷ دقیقاً همین را نشان می‌دهد. بن‌بست یکی از آن سه ویژگی نیست؛ شکست کل سیستم است وقتی چند قفل در کار باشد. یک مکانیزم می‌تواند

# چرا اجرای برنامه هیچ چیزی را اثبات نمی‌کند

## استدلال

- خطای هم‌روندی ممکن است برای ظاهر شدن به درهم‌تنیدگی خاصی از یک دوجین گام اتمی نیاز داشته باشد. یک
- آن در اختیار شما نیست، آن درهم‌تنیدگی ممکن است در یک میلیون اجرا رخ ندهد و بعد در تولید رخ بدهد. روی ماشینی که زمان‌بند
- بنابراین یک تست موفق فقط یعنی «هیچ نقضی مشاهده نشد»، هرگز «هیچ نقضی وجود ندارد».
- عکس آن هم برقرار است: یک تست ناموفق قطعی است، چون یک شاهد برای رد یک ویژگی ایمنی کافی است.

## این تمرین چه می‌کند

بحرانی نداشته باشد، انحصار متقابل برقرار است، تمام. هیچ اجرایی، هر قدر طولانی، چیزی به این گزاره نمی‌افزاید. بررسی‌گر مدل تمام فضای حالت دست‌یافتنی را می‌شمارد. اگر هیچ حالت دست‌یافتنی‌ای دو نخ را هم‌زمان در بخش به‌جای نمونه‌برداری از اجراها،

## تکرارپذیری اندازه‌گیری‌ها

مربوط به درستی از بررسی‌گر مدل می‌آیند و ادعاهای مربوط به هزینه از این میانه‌ها، و هر دو برچسب خورده‌اند. داشت؛ بنابراین هر زمان‌سنجی میانهٔ پنج تا هفت تکرار است و کمینه و بیشینه کنار آن گزارش شده است. ادعاهای شده‌اند و واقعاً پرنویزند. یک اجرای واحد از یک بار کاری یکسان روی ماشین این تمرین بیش از یک مرتبهٔ بزرگی تفاوت اعداد هزینه در نیمهٔ دوم این گزارش واقعاً اندازه‌گیری

> **:دو پرسش متفاوت**درست است؟» پاسخ می‌دهد و سنجش به «چه هزینه‌ای دارد؟». هیچ‌یک جایگزین دیگری نیست. دو نیمهٔ این تمرین به دو پرسش متفاوت پاسخ می‌دهند. بررسی‌گر مدل به «آیا

# بررسی‌گر مدل چگونه کار می‌کند

## مدل

```
state = ( shared-memory tuple,
          (pc0, private registers of thread 0),
          (pc1, private registers of thread 1) )

step(pid, pc, loc, shared) -> (pc', loc', shared', action)
    action in {None, 'enter', 'exit'}

successors(state) = one transition per thread that can move
```

مدل حالتی که برای پیمایش کامل به کار رفته است

## هر الگوریتم چه می‌دهد

- یک چندتایی از اعداد صحیح مشترک: پرچم‌ها، نوبت، سمافورها، صف‌های انتظار.
- یک تابع گام: یک عمل اتمی به‌ازای هر مقدار شمارندهٔ برنامه.
- یک محمول «در بخش بحرانی» برای آزمون انحصار متقابل.
- یک محمول «در حال تلاش» برای آزمون گرسنگی.

## مسدود شدن چگونه مدل می‌شود

هیچ جانشین مفیدی ندارد. همین است که بن‌بست را آشکار می‌کند: حالتی که تنها گذرهای خروجی‌اش خودحلقه‌اند. می‌شود که حالت را تغییر نمی‌دهد. پیمایش‌گر این خودحلقه را حذف می‌کند، بنابراین نخ مسدود ساده‌تر از این قرار نخ مسدود با گذاری مدل

## گرسنگی چگونه کشف می‌شود

- سپس محدود کردن آن به گذارهایی که در آن‌ها قربانی هرگز وارد نمی‌شود و هرگز از بخش ورود خارج نمی‌شود. ساخت گراف دست‌یافتنی،
- یافتن مؤلفه‌های قویاً همبند آن گراف محدودشده.
- هر مؤلفه‌ای که دوری داشته باشد که در آن نخ دیگر دست‌کم دو بار وارد شود، یک شاهد گرسنگی است.
- قربانی در آن دور دست‌کم یک گام بردارد: نخی که هرگز زمان‌بندی نشده گرسنه نیست، فقط اجرا نشده است. انصاف ضعیف با این شرط برقرار می‌شود که

# الگوریتم دکر

## پروتکل

```
flag[i] = True
while flag[j]:
    if turn == j:
        flag[i] = False
        while turn == j:      # wait for my turn
            pass
        flag[i] = True
# critical section
turn = j
flag[i] = False
# remainder section
```

الگوریتم دکر برای فرایند i با j = 1 - i

## چگونه کار می‌کند

- هر فرایند با بالا بردن پرچم خود علاقه‌اش را اعلام می‌کند.
- اگر فرایند دیگر علاقه‌مند نباشد، ورود فوری است.
- هر دو علاقه‌مند باشند، متغیر نوبت تساوی را می‌شکند: فرایندی که نوبت او نیست پرچم خود را پس می‌گیرد. اگر
- فرایندِ عقب‌نشسته منتظر می‌ماند تا نوبت به او برگردد، سپس دوباره اعلام و دوباره بررسی می‌کند.
- نوبت را به دیگری می‌سپارد و همین است که تضمین می‌کند دیگری برای همیشه پشت سر گذاشته نشود. هنگام خروج، فرایند

**ایدهٔ کلیدی:** را انجام می‌دهد. بدون عقب‌نشینی و واگذاری نوبت، پروتکل به همان گونهٔ معیوب صفحهٔ بعد فرو می‌ریزد. متغیر نوبت است که کار اصلی

# دکر: نتیجهٔ بررسی‌شده

## نتیجهٔ بررسی‌شده

| نتیجه | ویژگی |
|---|---|
| 324 | حالت‌های دست‌یافتنی |
| 648 | گذرها |
| برقرار | انحصار متقابل |
| بله | آزادی از بن‌بست |
| بله، برای هر دو فرایند | آزادی از گرسنگی |

جدول 2. نتیجهٔ بررسی مدل برای الگوریتم دکر

## چرا این نتیجه معتبر است

- فضای حالت به‌طور کامل پیمایش شده است، نه نمونه‌برداری.
- هیچ حالتی یافت نشد که در آن هر دو فرایند هم‌زمان در بخش بحرانی باشند.
- هیچ حالت دست‌یافتنی‌ای یافت نشد که در آن هر دو فرایند مسدود باشند.
- هیچ دور دست‌یافتنی‌ای یافت نشد که در آن یک فرایند بچرخد و دیگری پیاپی وارد شود.

# الگوریتم پترسون

## پروتکل

```
flag[i] = True
turn = j                      # be polite: give the turn away
while flag[j] and turn == j:  # wait only if both want in
    pass
# critical section
flag[i] = False
# remainder section
```

الگوریتم پترسون برای فرایند i با j = 1 - i

## تفاوت آن با دکر

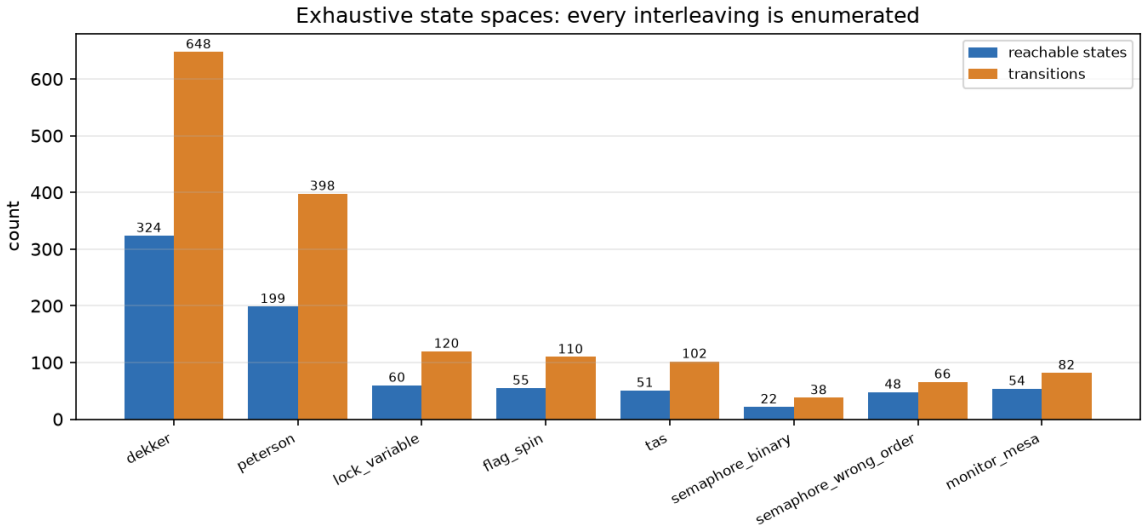
- در آن عقب‌نشینی و اعلام دوباره وجود ندارد؛ حلقه کوتاه‌تر است.
- نوبت را واگذار می‌کند و سپس منتظر می‌ماند، یعنی فرایند دیگر نوبت را به سود خود می‌یابد و پیش می‌رود. فرایند
- هر دو الگوریتم از دو پرچم و یک متغیر نوبت استفاده می‌کنند؛ پترسون آن‌ها را اقتصادی‌تر به کار می‌گیرد.
- پترسون از طریق درخت مسابقات به n فرایند تعمیم می‌یابد؛ دکر به‌سادگی تعمیم نمی‌یابد.

## نتیجهٔ بررسی‌شده

| نتیجه | ویژگی |
|---|---|
| 199 | حالت‌های دست‌یافتنی |
| 398 | گذرها |
| برقرار | انحصار متقابل |
| بله | آزادی از بن‌بست |
| بله، برای هر دو فرایند | آزادی از گرسنگی |

جدول 3. نتیجهٔ بررسی مدل برای الگوریتم پترسون

# چرا فضاهای حالت با هم فرق دارند




شکل 4. حالت‌ها و گذرهای دست‌یافتنی برای هر هشت الگوریتم، شمرده‌شده به‌صورت کامل

## خواندن نمودار

- دست‌یافتنی بیشتری دارد، چون حلقهٔ عقب‌نشینی و تلاشِ دوبارهٔ آن به شمارنده‌های برنامهٔ بیشتری نیاز دارد. دکر از پترسون حالت‌های
- حالت را دارد: مسدود شدن یک کلاس کامل از درهم‌تنیدگی‌ها را حذف می‌کند، چون نخ مسدود جانشینی ندارد. سمافور دودویی کوچک‌ترین فضای
- گونه‌های عمداً معیوب هم فضای کوچکی دارند، دقیقاً به این دلیل که گیر می‌کنند: رفتارهای دست‌یافتنی کمتر.
- است که به‌طور کامل پیمایش شود؛ هیچ چیز در اینجا به نمونه‌برداری یا به محدودیت عمق وابسته نیست. هر فضا آن‌قدر کوچک

# گونهٔ معیوب ۱: متغیر قفل تکی

## پروتکل وسوسه‌انگیز اما غلط

```
while flag[j]:                # look first
    pass
flag[i] = True                # then announce
# critical section
flag[i] = False
# remainder section
```

آزمون و سپس تنظیم روی یک متغیر: یک اشتباه کلاسیک

## چرا شکست می‌خورد

جداگانه‌اند. اگر هر دو فرایند پیش از آنکه هر یک بنویسد بخوانند، هر دو قفل را آزاد می‌بینند و هر دو وارد می‌شوند.
خواندن پرچم دیگر و نوشتن پرچم خود دو گام اتمی

## شاهد تولیدشده توسط بررسی‌گر

```
   pc=(0,0) shared=(0, 0) loc=((0,), (0,))
P0: pc=(1,0) shared=(0, 0) loc=((0,), (0,))
P0: pc=(2,0) shared=(0, 0) loc=((0,), (0,))
P1: pc=(2,1) shared=(0, 0) loc=((0,), (0,))
P0: pc=(3,1) shared=(1, 0) loc=((0,), (0,))
P0: pc=(4,1) shared=(1, 0) loc=((0,), (0,)) ENTER
P1: pc=(4,2) shared=(1, 0) loc=((0,), (0,))
P1: pc=(4,3) shared=(1, 1) loc=((0,), (0,))
P1: pc=(4,4) shared=(1, 1) loc=((0,), (0,)) ENTER
```

کوتاه‌ترین ردیابی نقض‌کننده از میان 60 حالت؛ ENTER یعنی ورود به بخش بحرانی

- هر دو فرایند flag[j] را برابر ۰ می‌خوانند در حالی که پرچم دیگر هنوز ۰ است.
- سپس هر دو پرچم خود را بالا می‌برند و وارد می‌شوند.
- دو سطر آخر هر دو به ENTER ختم می‌شوند: انحصار متقابل نقض شده است.
- این ردیابی حدس نیست؛ کوتاه‌ترین مسیری است که پیمایش‌گر تا یک حالت نقض‌کننده یافته است.

# گونهٔ معیوب ۲: اعلام و سپس چرخیدن

## پروتکل

```
flag[i] = True                # announce first
while flag[j]:                # then look
    pass
# critical section
flag[i] = False
# remainder section
```

جابه‌جایی دو گام، انحصار را درست می‌کند اما بن‌بست می‌سازد

## آنچه بررسی‌گر یافت

| نتیجه | ویژگی |
|---|---|
| 55 | حالت‌های دست‌یافتنی |
| برقرار | انحصار متقابل |
| **خیر** | آزادی از بن‌بست |
| بله (به‌طور بدیهی: کسی در انتظار گیر نمی‌کند) | آزادی از گرسنگی |

جدول 5. نتیجهٔ بررسی مدل برای گونهٔ اعلام-و-سپس-چرخیدن

## شاهد بن‌بست

```
   pc=(0,0) shared=(0, 0) loc=((0,), (0,))
P0: pc=(1,0) shared=(1, 0) loc=((0,), (0,))
P1: pc=(1,1) shared=(1, 1) loc=((0,), (0,))
```

هر دو فرایند علاقهٔ خود را اعلام کرده‌اند؛ اکنون هر یک تا ابد منتظر دیگری است

## خواندن ردیابی

- P0 سپس ،P1 پرچم ۱ را .پرچم ۰ را بالا می‌برد
- اکنون هر دو پرچم برقرارند و هر فرایند در آستانهٔ نگاه کردن است.
- از اینجا هیچ‌یک نمی‌تواند پرچم آزادی ببیند، بنابراین هیچ‌یک پیش نمی‌رود.

# اعلام-و-سپس-چرخیدن: چرا ترتیب دو گام مهم است

## چرا شکست می‌خورد

- اعلام پیش از نگاه کردن، انحصار را تضمین می‌کند: هرکس پرچم دیگر را آزاد ببیند در امان است.
- اما اگر هر دو پیش از آنکه هر یک نگاه کند اعلام کنند، هر دو تا ابد می‌چرخند.
- این یک شکستِ پیشرفت است، نه شکست ایمنی: هیچ اتفاق بدی نمی‌افتد، چون اصلاً هیچ اتفاقی نمی‌افتد.
- هر دو با افزودن یک تساوی‌شکن که یکی از دو طرف می‌تواند در آن ببرد، از این وضعیت جلوگیری می‌کنند.

دکر و پترسون

## درس کلی

کوتاه تست دیده نمی‌شود: اولی به یک درهم‌تنیدگی خوش‌شانس نیاز دارد و دومی به رسیدن هم‌زمان هر دو فرایند. را از دست می‌دهد به پروتکلی تبدیل شود که پیشرفت را از دست می‌دهد. هیچ‌یک از دو شکست در یک اجرای و یک نوشتن از متغیرهای مشترک بستگی دارد. جابه‌جا کردن دو خط مجاور کافی است تا پروتکلی که انحصار متقابل درستی در اینجا به ترتیب نسبی یک خواندن

| آزادی از بن‌بست | انحصار متقابل | گونه |
|---|---|---|
| بله | **نقض‌شده** | نگاه و سپس اعلام |
| **خیر** | برقرار | اعلام و سپس نگاه |
| بله | برقرار | دکر (اعلام + نوبت) |

جدول 6. سه چیدمان از یک ایدهٔ واحد

# انحصار درست، انتظار نامحدود :Test-and-Set

## پریمتیو

```
def TAS(lock):                # atomic read-modify-write
    old = lock
    lock = True
    return old

while TAS(lock):
    pass                      # spin
# critical section
lock = False
```

یک قفل چرخشی مبتنی بر Test-and-Set

## نتیجهٔ بررسی‌شده

| نتیجه | ویژگی |
|---|---|
| 51 | حالت‌های دست‌یافتنی |
| برقرار | انحصار متقابل |
| بله | آزادی از بن‌بست |
| **خیر، برای هر دو فرایند** | آزادی از گرسنگی |

جدول 7. نتیجهٔ بررسی مدل برای قفل Test-and-Set

## چرا انتظار محدود برقرار نیست

است. بررسی‌گر دوری دست‌یافتنی نشان می‌دهد که در آن یک فرایند می‌چرخد در حالی که دیگری پیاپی وارد می‌شود. از آنکه یک فرایند قفل را آزاد می‌کند، هر چرخنده‌ای می‌تواند در مسابقه ببرد، از جمله کسی که یک لحظه پیش رسیده هیچ چیز در این پروتکل به خاطر نمی‌سپارد چه کسی طولانی‌تر منتظر مانده است. پس

- انحصار متقابل با اتمی بودن TAS تضمین می‌شود.
- پیشرفت برقرار است: اگر قفل آزاد باشد، کسی آن را می‌گیرد.
- انتظار محدود برقرار نیست: یک فرایند می‌تواند تا ابد پشت سر گذاشته شود.
- به همین دلیل قفل‌های چرخشی واقعی یک صف می‌افزایند (قفل نوبتی، قفل MCS).

> **چرا اینجا آمده است:** Test-and-Set مستقل‌اند واقعاً محدود نه. این روشن‌ترین نمایش آن است که آن سه ویژگی برقرارند اما انتظار. تنها مکانیزمی در این تمرین است که در آن انحصار متقابل و آزادی از بن‌بست

# سمافورها

## تعریف

پروتکل‌های نرم‌افزاری بالا، به دو شرکت‌کننده محدود نیست و نخ مسدود نمی‌چرخد: می‌خوابد و بیدار می‌شود.
سمافور یک عدد صحیح با دو عمل اتمی است. برخلاف

```
wait(S)   / acquire:  if S > 0 then S = S - 1 else block
```

عمل گرفتن

```
signal(S) / release:  if a thread is queued, wake one; else S = S + 1
```

عمل آزاد کردن

## پیاده‌سازی از پایه در این تمرین

```
class Semaphore:
    def __init__(self, value):
        self.value = value
        self._cv = threading.Condition()

    def acquire(self):
        with self._cv:
            while self.value == 0:   # must be a loop
                self._cv.wait()
            self.value -= 1

    def release(self):
        with self._cv:
            self.value += 1
            self._cv.notify()
```

یک سمافور شمارنده پیاده‌شده روی یک متغیر شرطی

# سمافورها: نتیجهٔ بررسی‌شده

## نتیجهٔ بررسی‌شده

| نتیجه | ویژگی |
|---|---|
| 22 | حالت‌های دست‌یافتنی |
| 38 | گذرها |
| برقرار | انحصار متقابل |
| بله | آزادی از بن‌بست |
| بله (صف انتظار FIFO است) | آزادی از گرسنگی |

جدول 8. نتیجهٔ بررسی مدل برای یک سمافور دودویی با استفادهٔ درست

- FIFO است که انتظار محدود را می‌خرد: یک نخ نمی‌تواند توسط کسی که دیرتر رسیده پشت سر گذاشته شود. این صف انتظار
- سمافور از دکر و پترسون عمومی‌تر است: هر تعداد شرکت‌کننده و هر تعداد منبع را پشتیبانی می‌کند.
- اما استفادهٔ نادرست از آن هم آسان‌تر است که موضوع صفحهٔ بعد است.

## سمافور شمارنده در برابر دودویی

- سمافور دودویی فقط ۰ و ۱ می‌گیرد و برای انحصار متقابل به کار می‌رود.
- سمافور شمارنده پی می‌گیرد چند واحد از یک منبع آزاد است؛ مثلاً برای استخری از اتصال‌ها.
- پیاده‌سازی بالا یک سمافور شمارنده است؛ مقدار اولیهٔ ۱ آن را دودویی می‌کند.
- نیاز دارند گرفتن و آزاد کردن درست جفت شوند؛ یک آزاد کردنِ اضافه بی‌سر و صدا اجازهٔ ورود دو نخ را می‌دهد. هر دو

# گونهٔ معیوب ۳: انتظار دوره‌ای

## دو قفل، دو ترتیب

```
# thread 0                # thread 1
lock(A)                   lock(B)
lock(B)                   lock(A)     # opposite order
# critical section        # critical section
unlock(B)                 unlock(A)
unlock(A)                 unlock(B)
```

روش کلاسیک بن‌بست ساختن با پریمتیوهای درست

## نتیجهٔ بررسی‌شده

| نتیجه | ویژگی |
|---|---|
| 48 | حالت‌های دست‌یافتنی |
| برقرار | انحصار متقابل |
| **خیر** | آزادی از بن‌بست |
| بله | آزادی از گرسنگی |

جدول 9. نتیجهٔ بررسی مدل برای گونهٔ ترتیبِ معکوس

## اندازه‌گیری روی نخ‌های واقعی

```
wrong order    : holds A, blocked on B | holds B, blocked on A
                 deadlocked = True
same order     : completed | completed
                 completed  = True
```

خروجی آزمایش قطعی بن‌بست

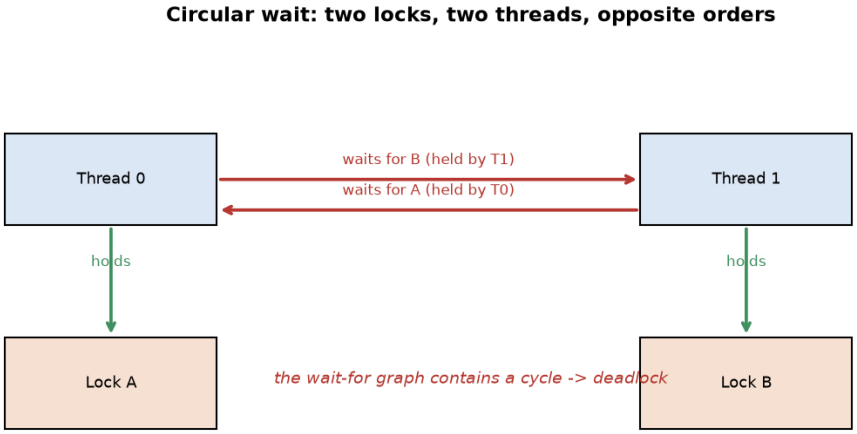
- مانع میان دو گرفتن قرار دارد، بنابراین انتظار دوره‌ای به‌طور عمدی ساخته می‌شود نه به شانس زمان‌بند واگذار. یک
- گرفتنِ دوم غیرمسدودکننده است، بنابراین نتیجه نمی‌تواند به مسابقهٔ زمان‌سنجی وابسته باشد.
- با ترتیب یکسان، همان برنامه همیشه کامل می‌شود.

# بن‌بست: چهار شرط و راه‌های شکستن آن‌ها

## چهار شرط کافمن

| شرط | معنا در این مثال | برقرار؟ |
|---|---|---|
| انحصار متقابل | یک قفل در اختیار حداکثر یک نخ است | بله |
| نگه‌داشتن و انتظار | نخی A را دارد و منتظر B است | بله |
| غیرقابل‌سلب بودن | قفل را نمی‌توان از کسی گرفت | بله |
| انتظار دوره‌ای | T0 منتظر T1 و T1 منتظر T0 | بله |

جدول 10. هر چهار شرط برای بن‌بست لازم‌اند




شکل 11. گراف انتظار مثال ترتیبِ معکوس؛ دور موجود در آن همان بن‌بست است

# راهبردهای پیشگیری از بن‌بست

## راهبردهای پیشگیری

- **تحمیل یک ترتیب سراسری قفل.**شود؛ دور ناممکن می‌شود. ارزان‌ترین اصلاح و همان که بالا اندازه‌گیری شد. همه‌جا A پیش از B گرفته
- **گرفتن همه با هم.**گرفتن همهٔ قفل‌ها در یک گام همه یا هیچ، یا هیچ‌کدام.
- **اجازهٔ سلب.**خورد، آنچه داری آزاد کن و دوباره تلاش کن. بدون عقب‌نشینی تدریجی خطر زنده‌بست دارد. اگر گرفتن دوم شکست
- **کشف و بازیابی.**اجرا و حذف یک قربانی. در پایگاه داده‌ها به کار می‌رود و در هستهٔ سیستم‌عامل نادر است. ساخت گراف انتظار در زمان

**درس:**استفاده از قفل‌هاست، نه نحوهٔ پیاده‌سازی آن‌ها. یک سمافور بی‌نقص هم به همان آسانی بن‌بست می‌سازد. هیچ‌یک از چهار شرط دربارهٔ کیفیت قفل نیست. بن‌بست ویژگیِ نحوهٔ

# مانیتورها

## تعریف

شرطی به یک نخ اجازه می‌دهند تا برقرار شدن یک گزاره منتظر بماند و در این مدت قفل مانیتور را آزاد کند.
روی آن در یک بسته می‌گذارد و تضمین می‌کند در هر لحظه حداکثر یک نخ درون مانیتور در حال اجرا باشد. متغیرهای مانیتور وضعیت مشترک را همراه با عملیات

## معنای Hoare در برابر Mesa

| (استفاده‌شده در اینجا) Mesa | Hoare | جنبه |
|---|---|---|
| می‌شود، فرستنده به اجرا ادامه می‌دهد منتظر آماده | می‌شود، منتظر بلافاصله اجرا می‌شود فرستنده مسدود | هنگام سیگنال |
| باید دوباره بررسی شود | تضمیناً درست است | گزاره هنگام بیداری |
| `while not P: wait(c)` | `if not P: wait(c)` | حلقهٔ درست |
| ساده‌تر، استاندارد در جاوا و پایتون | سخت‌تر، تعویض زمینهٔ بیشتر | پیاده‌سازی |

جدول 12. دو معنای مانیتور و دلیل نیاز Mesa به حلقهٔ while

## چرا حلقهٔ while اختیاری نیست

ممکن است نخ دیگری گزاره را باطل کرده باشد. بنابراین حلقه بخشی از استدلال درستی است، نه یک احتیاطِ سبکی.
تحت معنای Mesa، نخِ بیدارشده فقط آماده می‌شود؛ باید دوباره قفل مانیتور را بگیرد، و تا آن زمان

```
def put(self, item):
    with self._cv:
        while len(self.buffer) >= self.capacity:   # loop
            self._cv.wait()
        self.buffer.append(item)
        self._cv.notify_all()
```

سویهٔ تولیدکننده در مانیتور بافر محدود

# اندازه‌گیری: مانیتور بافر محدود

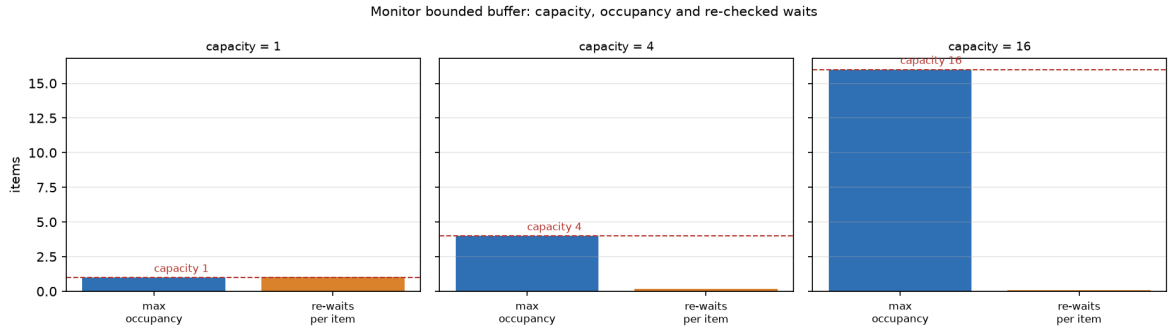
## آزمایش

پایه روی یک متغیر شرطی پیاده‌سازی شده است. هر پیکربندی پنج بار اجرا شد و سریع‌ترین اجرا گزارش شده است. دو تولیدکننده و دو مصرف‌کننده، ۱۶۰۰ قلم را از بافری با ظرفیت ۱، ۴ و ۱۶ عبور می‌دهند. مانیتور از

| توان (قلم بر ثانیه) | دوباره پس از بیداری بررسی | رعایت ظرفیت | بیشینهٔ اشغال | ظرفیت |
|---|---|---|---|---|
| 23,343 | 1,383 | بله | 1 | 1 |
| 51,045 | 361 | بله | 4 | 4 |
| 114,212 | 147 | بله | 16 | 16 |

جدول 13. نتایج تولیدکننده/مصرف‌کننده؛ هر بار ۱۶۰۰ قلم

# بافر محدود: هزینهٔ بررسی دوباره




شکل 14. بیشینهٔ اشغال و تعداد دفعاتی که یک نخ بیدارشده دوباره منتظر ماند تا گزاره برقرار شود

- تولید و مصرف در هر اجرا کاملاً متوازن بود: هیچ قلمی گم یا تکرار نشد.
- اشغال بافر هرگز از ظرفیت فراتر نرفت.
- در ظرفیت ۱، یک نخ بیدارشده 1,383 بار ناچار شد دوباره منتظر بماند، در برابر تنها 147 بار در ظرفیت ۱۶.
- این عدد گواه مستقیم آن است که حلقهٔ while کار واقعی انجام می‌دهد: اگر کد از `if` وضعیت تبدیل می‌شدند. استفاده کرده بود، این انتظارهای دوباره به جای آن به خرابیِ بی‌صدای
- بافر بزرگ‌تر توان را 4.9 برابر می‌کند، چون تولیدکننده‌ها کمتر مسدود می‌شوند.

# مسئلهٔ خوانندگان و نویسندگان

## مسئله

انحصاری نیاز دارد. پرسش این است که وقتی خواننده‌ای درون است و نویسنده‌ای می‌رسد، یا برعکس، چه باید کرد.
را می‌خوانند و برخی در آن می‌نویسند. هر تعداد خواننده می‌تواند هم‌زمان درون باشد، اما نویسنده به دسترسی
نخ‌های بسیاری یک شیء مشترک

## سه راه‌حل پیاده‌شده در اینجا

| پیامد | قاعده | گونه |
|---|---|---|
| نویسندگان می‌توانند گرسنه بمانند | وارد می‌شود مگر نویسنده‌ای داخل باشد خوانندهٔ جدید | اولویت خوانندگان |
| خوانندگان می‌توانند گرسنه بمانند | منتظر، خوانندگان جدید را مسدود می‌کند نویسندهٔ | اولویت نویسندگان |
| کسی گرسنه نمی‌ماند؛ هم‌روندی کمتر | هر که زودتر رسیده نخست سرو می‌شود | منصفانه (نوبت FIFO) |

جدول 15. سه گونه و بده‌بستان انصاف در آن‌ها

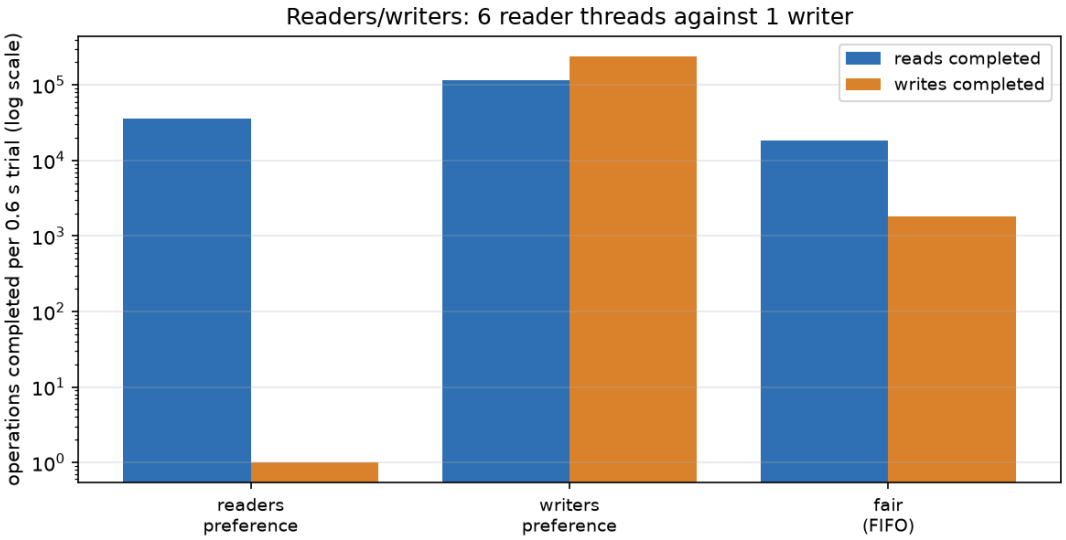
## اندازه‌گیری: ۶ نخ خواننده در برابر ۱ نویسنده

| میانگین سهم خواندن | نوشتن‌های کامل‌شده | خوانش‌های کامل‌شده | گونه |
|---|---|---|---|
| 1.000 | 5 | 189,832 | اولویت خوانندگان |
| 0.292 | 1,165,632 | 479,104 | اولویت نویسندگان |
| 0.898 | 10,136 | 91,051 | منصفانه (FIFO) |

جدول 16. پنج آزمایش ۰٫۶ ثانیه‌ای، تجمیع‌شده

# خوانندگان و نویسندگان: خواندن نتایج

- عمل انجام داد در حالی که خواننده‌ها 189,832 عمل انجام دادند: نویسنده عملاً هر بار پشت سر گذاشته شد. تحت اولویت خوانندگان، نویسنده در مجموع تنها 5
- تحت اولویت نویسندگان نابرابری معکوس می‌شود: 1,165,632 نوشتن در برابر 479,104 خواندن.
- نزدیک به نسبت ورودِ ۶ خواننده به ۱ نویسنده (۰٫۸۵۷)، که همان چیزی است که انصاف باید تولید کند. گونهٔ FIFO به سهم خواندنِ 0.898 می‌رسد،




شکل 17. عملیات کامل‌شده در هر آزمایش، مقیاس لگاریتمی؛ فاصلهٔ دو ستون همان کسر انصاف است

## خواندن نمودار

- دو ستونِ گونهٔ اولویت خوانندگان چهار مرتبهٔ بزرگی با هم فاصله دارند.
- دو ستونِ گونهٔ FIFO تقریباً به نسبت تعداد نخ‌ها هستند، که همان نتیجهٔ منصفانهٔ مورد انتظار است.
- گونهٔ اولویت نویسندگان تصویر آینه‌ای است: ستون نوشتن غلبه دارد.

**خواندن صادقانه:** به این دلیل گزارش شده‌اند که بده‌بستان را ملموس کنند، نه اینکه چیزی دربارهٔ گرسنگی اثبات کنند. این اعداد رفتار واقعیِ کد را نشان می‌دهند، نه یک قضیه. آن‌ها

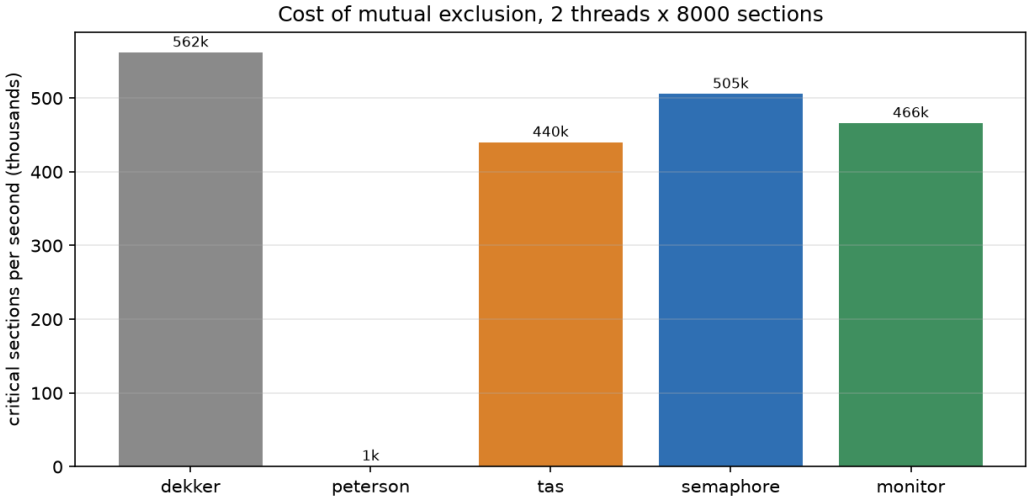
# درستی چه هزینه‌ای دارد

## آزمایش

شمارندهٔ مشترک را افزایش می‌دهند. هفت تکرار؛ میانه گزارش شده است همراه با کمینه و بیشینهٔ مشاهده‌شده. دو نخ هر کدام ۵۰۰۰ بخش بحرانی اجرا می‌کنند و یک

| مکانیزم | میانه (عمل بر ثانیه) | کمینه | بیشینه | چرخش‌ها (میانه) |
|---|---|---|---|---|
| dekker | 995,696 | 245,408 | 1,029,634 | 0 |
| monitor | 551,455 | 528,263 | 557,830 | ندارد (مسدود می‌شود) |
| peterson | 1,048,660 | 1,018,671 | 1,069,822 | 0 |
| semaphore | 501,066 | 477,697 | 527,004 | ندارد (مسدود می‌شود) |
| tas | 425,099 | 408,594 | 745,474 | 37,613 |

جدول 18. توانِ پنج مکانیزم انحصار متقابل




شکل 19. توانِ میانه؛ بیشتر یعنی سریع‌تر

# تفسیر اعداد توان

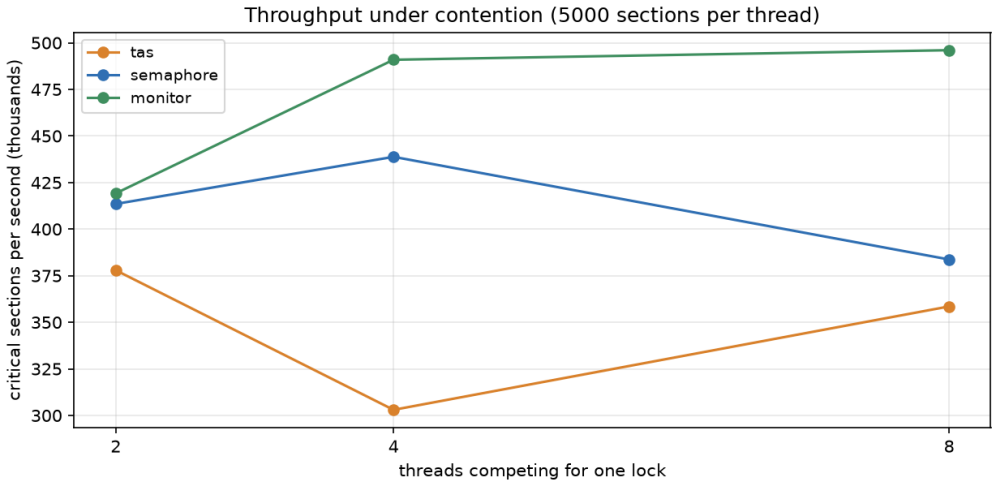
## اعداد چه می‌گویند

- دو پروتکل نرم‌افزاری در اینجا سریع‌ترند چون هرگز نمی‌خوابند: نخ یا می‌چرخد یا وارد می‌شود.
- سرعت دارند و این تفاوت بهای مسدود شدن است: یک انتظار و بیداری در هسته به‌ازای هر گرفتنِ مورد رقابت. سمافور و مانیتور تقریباً نصفِ آن
- میان کمینه و بیشینهٔ اجراها بزرگ است؛ روی یک ماشین اشتراکی یک اندازه‌گیری واحد قابل اعتماد نیست. فاصلهٔ
- هر پنج مکانیزم شمارندهٔ درست تولید کردند و هرگز اجازه ندادند دو نخ هم‌زمان وارد بخش بحرانی شوند.

# رفتار در شرایط رقابت

| تعداد نخ‌ها | Test-and-Set (عمل/ثانیه) | سمافور (عمل/ثانیه) | مانیتور (عمل/ثانیه) | چرخش‌های TAS |
|---|---|---|---|---|
| 2 | 554,397 | 524,972 | 540,953 | 24,163 |
| 4 | 624,466 | 492,372 | 532,148 | 24,714 |
| 8 | 476,978 | 494,447 | 510,747 | 141,814 |

جدول 20. میانهٔ پنج آزمایش، ۵۰۰۰ بخش بحرانی به‌ازای هر نخ

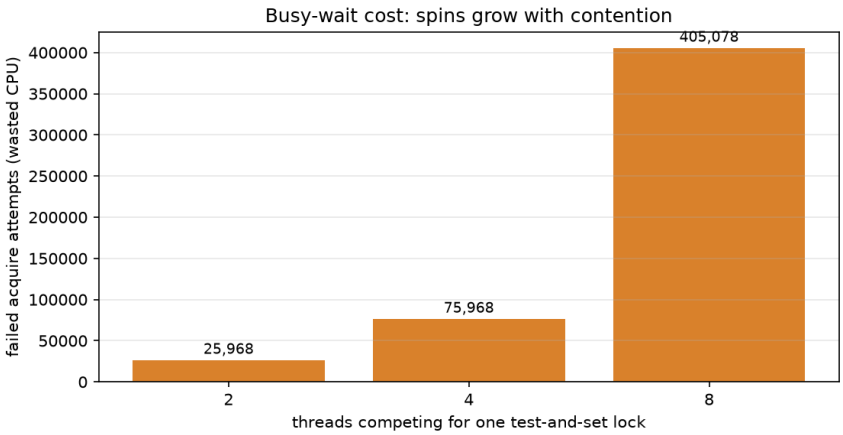



شکل 21. توان با افزایش تعداد نخ‌های رقیب

## تفسیر

- به‌نرمی افت می‌کنند: مسدود کردن یک نخ، پردازنده را برای کسی که می‌تواند اجرا شود آزاد می‌گذارد. مکانیزم‌های مسدودکننده
- تفاوت میان سه مکانیزم در برابر گستردگیِ میان اجراهای تکراری یک مکانیزم واحد کوچک است.
- قفل چرخشی از نظر توان فرو نمی‌پاشد، چون بخش بحرانی در اینجا به‌شدت کوتاه است: یک افزایش.

# هزینهٔ پنهان چرخیدن




شکل 22. تلاش‌های ناموفق Test-and-Set: کاری که انجام و دور انداخته می‌شود

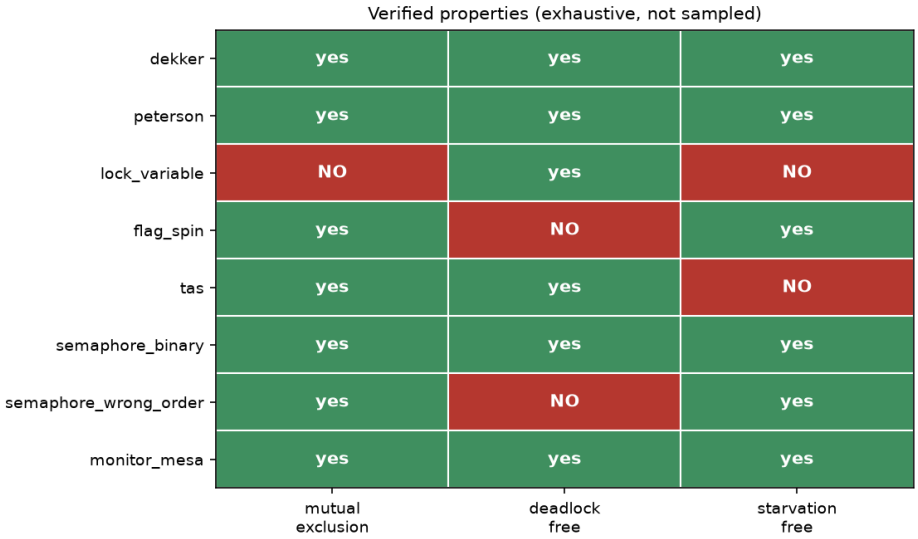
- هر تلاش ناموفق یک خواندن-تغییر-نوشتنِ هدررفته است.
- این شمارش با تعداد نخ‌های رقیب به‌شدت رشد می‌کند، چون نخ‌های بیشتری بر سر یک کلمه می‌جنگند.
- بخش بحرانی از هزینهٔ تعویض زمینه کوتاه‌تر باشد؛ این در درون هسته درست است و در کد کاربردی به‌ندرت. قفل چرخشی تنها زمانی انتخاب درست است که
- اگر نخِ دارندهٔ قفل از زمان‌بندی خارج شود، هر نخ دیگر یک برش زمانیِ کامل را بی‌هوده می‌سوزاند.

# جدول مقایسه

| الگوریتم | خانواده | تعداد حالت‌ها | انحصار متقابل | آزادی از بن‌بست | آزادی از گرسنگی |
|---|---|---|---|---|---|
| `dekker` | نرم‌افزاری | 324 | برقرار | بله | بله |
| `peterson` | نرم‌افزاری | 199 | برقرار | بله | بله |
| `lock_variable` | معیوب | 60 | **نقض‌شده** | بله | **خیر** |
| `flag_spin` | معیوب | 55 | برقرار | **خیر** | بله |
| `tas` | سخت‌افزاری | 51 | برقرار | بله | **خیر** |
| `semaphore_binary` | پریمتیو | 22 | برقرار | بله | بله |
| `semaphore_wrong_order` | معیوب | 48 | برقرار | **خیر** | بله |
| `monitor_mesa` | مانیتور | 54 | برقرار | بله | بله |

جدول 23. هر نتیجه در این جدول کامل است، نه نمونه‌برداری‌شده

# جدول مقایسه به صورت ماتریس ویژگی‌ها




شکل 24. همان نتایج در قالب یک ماتریس ویژگی

## چگونه آن را بخوانیم

- می‌سازد، یکی نه انحصار را غلط برقرار می‌کند و نه بن‌بست می‌سازد اما سبقت بی‌کران را ممکن می‌گذارد.
سه مورد از هشت مورد معیوب‌اند، و هر یک به شیوه‌ای متفاوت: یکی انحصار را از دست می‌دهد، یکی بن‌بست
- دو پروتکل نرم‌افزاری درست و دو مکانیزم سطح‌بالاتر درست، هر سه ویژگی را دارند.
- سمافوری با ترتیبِ اشتباه همهٔ ویژگی‌های خودِ پریمتیو را نگه می‌دارد و باز هم سیستم را بن‌بست می‌دهد.

# چه زمانی از چه چیزی استفاده کنیم

| چرا | توصیه | وضعیت |
|---|---|---|
| کوتاه‌ترین پروتکل نرم‌افزاریِ درست | پترسون | بدون پشتیبانی سیستم‌عامل، آموزش دو فرایند، |
| پروتکل‌های دوفرایندی کاربرد ندارند | سمافور یا مانیتور | بیش از دو شرکت‌کننده |
| مسدود شدن بیش از انتظار هزینه دارد | قفل چرخشی (با صف) | بخش بحرانی کوتاه‌تر از تعویض زمینه |
| پردازنده را با چرخیدن نسوزانید | سمافور یا مانیتور مسدودکننده | بخش بحرانی کار واقعی انجام می‌دهد |
| انحصار و انتظار را هم‌زمان محصور می‌کند | مانیتور با متغیرهای شرطی | مشترک به همراه گزاره‌ای برای انتظار وضعیت |
| در صفحهٔ ۱۷ جلوگیری می‌کند از انتظار دوره‌ایِ اندازه‌گیری‌شده | هر کدام، با یک ترتیب سراسری | نیاز به چند قفل |
| اولویت‌دار یک سمت را گرسنه می‌گذارند گونه‌های | خواننده/نویسندهٔ منصفانه | دادهٔ عمدتاً خواندنی |

جدول 25. یک راهنمای تصمیم عملی

## آنچه این گزارش توصیه نمی‌کند

- آن‌ها تنها تحت سازگاری ترتیبی درست‌اند، که سخت‌افزار امروزی بدون موانع حافظهٔ صریح به شما نمی‌دهد. استفاده از دکر یا پترسون در کد تولید.
- ساختن قفلِ خود در کد کاربردی، هر وقت زمان‌اجرا یکی فراهم می‌کند.
- فرض اینکه سمافور از مانیتور «ایمن‌تر» است. سمافور پریمتیوتر است، یعنی راه‌های بیشتری برای اشتباه دارد.

# محدودیت‌ها و دامنهٔ صادقانه

## چه چیزی اثبات شده است

- برای هر یک از هشت الگوریتم، نتیجه‌ای روی تمام فضای حالت دست‌یافتنی، تحت مدلِ صفحهٔ ۷.
- انحصار متقابل، آزادی از بن‌بست و آزادی از گرسنگی به‌طور کامل تعیین تکلیف شده‌اند، نه نمونه‌برداری‌شده.

## چه چیزی اثبات نشده است

- اند، اما نتایج سمافور و مانیتور به‌طور خودکار به n نخ تعمیم نمی‌یابد: در اینجا فقط حالتِ دو نخ بررسی شد. تنها دو شرکت‌کننده مدل شده‌اند. دکر و پترسون الگوریتم‌های دوفرایندی
- بکند. روی سخت‌افزار واقعی، دکر و پترسون به موانع حافظه نیاز دارند که این مدل آن‌ها را نمایش نمی‌دهد. را فرض می‌کند. سخت‌افزار واقعی خواندن‌ها و نوشتن‌ها را بازمرتب می‌کند و کامپایلر هم همین کار را می‌تواند مدل سازگاری ترتیبی
- در مدل خرابی، زمان‌سنجی و وقفه وجود ندارد.
- مدل مانیتور یک بافر تک‌خانه با یک تولیدکننده و یک مصرف‌کننده است؛ بررسی‌گر عمومی مانیتور نیست.

## اندازه‌گیری‌ها چه نشان می‌دهند و چه نشان نمی‌دهند

- زمان‌اجرا در یک محیط اشتراکی دارای GIL هستند. نسبت هزینه را نشان می‌دهند، نه یک سنجش قابل انتقال. این‌ها زمان‌سنجی‌های یک ماشین و یک
- برابر زمان‌اجرای بومی به‌شدت تغییر می‌دهد؛ شمارش چرخش‌ها بیش از همه تحت تأثیر است GIL ِCPython .رفتار رقابت را در
- خواننده/نویسنده رفتار واقعیِ کد است، اما نسبت‌های دقیق به پیاده‌سازی قفل در زمان‌اجرا وابسته است. گرسنگیِ

**قدرت ادعاها:** ۷ است. ضعیف‌ترین ادعاها زمان‌سنجی‌ها هستند، و به همین دلیل میانه‌هایی با گسترهٔ گزارش‌شده‌اند. قوی‌ترین ادعای این گزارش جدول بررسی مدل است، و آن هم مشروط به مدلِ صفحهٔ

# نتیجه‌گیری و پاسخ‌ها

درسی هم‌خوان‌اند، و جایی که متن درسی معمولاً مبهم است، این گزارش به‌جای ادعا یک ردیابی شاهد ارائه می‌کند. بررسی شد و پنج مکانیزم روی نخ‌های واقعی اندازه‌گیری شد. نتایج بررسی‌شده و اندازه‌گیری‌شده با گزارش متون هشت الگوریتم انحصار متقابل به‌طور کامل

- دکر و پترسون هر سه ویژگی را برای دو فرایند فراهم می‌کنند، به بهای فرض سازگاری ترتیبی.
- محدود را نه: بررسی‌گر دوری نشان می‌دهد که در آن یک نخ تا ابد پشت سر گذاشته می‌شود Test-and-Set .انحصار و پیشرفت را فراهم می‌کند اما انتظار
- سمافور دودویی با صف FIFO هر سه را فراهم می‌کند و فراتر از دو شرکت‌کننده هم تعمیم می‌یابد.
- شرطی می‌افزاید و اندازه‌گیری نشان می‌دهد که حلقهٔ while در ظرفیت ۱، 1,383 بار واقعاً لازم شد. مانیتور Mesa همگام‌سازی
- همان سمافوری که درست بررسی می‌شود، وقتی دو نخ دو قفل را با ترتیب معکوس می‌گیرند بن‌بست می‌دهد. بن‌بست خطای استفاده است، نه نقص پریمتیو:

## پاسخ به پرسش‌های تمرین

| پاسخ | پرسش |
|---|---|
| بله، به‌طور کامل برای دو فرایند بررسی شد | آیا دکر و پترسون انحصار متقابل را تضمین می‌کنند؟ |
| بله، هر دو انتظار محدود دارند | آیا هر یک از آن‌ها آزاد از گرسنگی است؟ |
| خیر؛ این تنها ویژگی failing آن است | آیا Test-and-Set انتظار محدود دارد؟ |
| بله؛ آزمایش ترتیبِ معکوس آن را قطعی نشان می‌دهد | آیا یک پریمتیو درست باز هم می‌تواند بن‌بست بدهد؟ |
| قفل را بگیرد و ممکن است گزاره در این میان باطل شده باشد نخ بیدارشده باید دوباره | چرا مانیتور Mesa باید گزاره‌اش را دوباره بررسی کند؟ |
| هرگز مسدود نمی‌شوند؛ اما تحت رقابت پردازنده می‌سوزانند پروتکل‌های چرخشی، چون | کدام در اینجا سریع‌تر است؟ |

جدول 26. پاسخ‌های کوتاه

# منابع و دستورهای بازتولید

## منابع


- Dijkstra, E. W. — Solution of a problem in concurrent programming control — Communications of the ACM, 1965. (الگوریتم دکر به روایت دیکسترا.)
- Peterson, G. L. — Myths About the Mutual Exclusion Problem — Information Processing Letters, 1981.
- Dijkstra, E. W. — The structure of the THE multiprogramming system — Communications of the ACM, 1968. (سمافورها.)
- Hoare, C. A. R. — Monitors: An Operating System Structuring Concept — Communications of the ACM, 1974.
- Lampson, B. W., & Redell, D. D. — Experience with Processes and Monitors in Mesa — Communications of the ACM, 1980. (معنای Mesa.)
- Coffman, E. G., Elphick, M. J., & Shoshani, A. — System Deadlocks — Computing Surveys, 1971.
- Silberschatz, A., Galvin, P. B., & Gagne, G. — Operating System Concepts — فصل‌های Process Synchronization و Deadlocks.


## دستورهای بازتولید

```
# از ریشهٔ خانه
python code/sync_model.py          # همهٔ تست‌ها را اجرا می‌کند
python code/run_experiments.py     # فقط آزمایش‌ها را دوباره می‌سازد
python code/make_diagrams.py       # فقط شکل‌ها را می‌کشد
python -m pytest tests -q          # 52 تست
```

هر چه در این گزارش هست با یک دستور بازتولید می‌شود

> **منشأ:** می‌آید که توسط run_experiments.py نوشته می‌شود. هیچ چیزی با دست تایپ نشده است.
> هر شکل و هر عددِ نقل‌شده در این گزارش از output/experiments.json

# فهرست فایل‌ها

| محتوا | مسیر |
|---|---|
| مستندات این تمرین | README.md |
| بررسی‌گر مدل: ۸ الگوریتم، نتایج و ردیابی‌های شاهد | code/sync_model.py |
| TAS، سمافور، مانیتور، خواننده/نویسنده و آزمایش بن‌بست پیاده‌سازی اجرایی دکر، پترسون، | code/sync_algorithms.py |
| نوشتن همهٔ خروجی‌ها در output/ | code/run_experiments.py |
| بازسازی همهٔ شکل‌ها | code/make_diagrams.py |
| نمایشِ پیشینِ سمافور و مانیتور | code/sync_demo.py |
| قفل کردن نتایج بررسی‌شده | tests/test_sync_model.py |
| تست لایهٔ نخی | tests/test_sync_algorithms.py |
| تست نمایش پیشین | tests/test_sync_demo.py |
| هر عددِ اندازه‌گیری‌شده | output/experiments.json |
| نتایج به همراه ردیابی‌های شاهد | output/model_check.txt |
| گزارش متنی کامل | output/experiments_summary.txt |
| ۸ شکلِ استفاده‌شده در این گزارش | diagrams/ |
| matplotlib و pytest | requirements.txt |
| دور نگه داشتن بایت‌کد از شاخه | .gitignore |

جدول 27. آنچه روی این شاخه منتشر می‌شود

# فهرست شکل‌ها و بازتولید کامل

## فهرست شکل‌ها

| نمایش | شکل |
|---|---|
| حالت‌های دست‌یافتنی هر الگوریتم | fig_state_spaces.png |
| سه ویژگی در قالب ماتریس | fig_property_matrix.png |
| هزینهٔ هر مکانیزم | chart_throughput.png |
| توان در برابر تعداد نخ‌ها | chart_contention.png |
| کار هدررفته در قفل چرخشی | chart_spins.png |
| انصافِ سه گونه | chart_readers_writers.png |
| انتظار دوره‌ای در قالب گراف انتظار | fig_deadlock.png |
| اشغال بافر و انتظارهای بررسی‌شدهٔ دوباره | fig_monitor_timeline.png |

جدول 28. هشت شکل این گزارش

## بازتولید کامل

- هر عدد از output/experiments.json خوانده می‌شود.
- هر شکل توسط code/make_diagrams.pyاز همان توابع اندازه‌گیری ساخته می‌شود.
- است، بنابراین اگر کسی تابع گام یک الگوریتم را تغییر دهد و ویژگی‌ای را بشکند، تست‌ها ناموفق می‌شوند.
- هر ادعای درستی توسط tests/test_sync_model.py قفل شده